
Guía de Ejercicios: Lenguaje C

EIE 434 – Programación 2 | Escuela de Ingeniería Eléctrica

Temática: Funciones, Ciclos y Condicionales **Semestre:** 1er Semestre 2026 **Docente:** Guillermo Cid Ampuero

Ayudante: Amaro Díaz

1. Objetivos

Esta guía tiene como objetivo aplicar la modularización del código mediante la creación y uso de **funciones**, integrando de manera simultánea el uso de **ciclos iterativos** y **estructuras condicionales** en problemas aplicados a la ingeniería.

2. Ejercicios

Ejercicio 1 – Monitoreo de Temperatura de un Motor

Contexto: En aplicaciones de robótica industrial, es vital monitorear la temperatura de los actuadores para evitar daños térmicos.

Instrucciones:

1. Cree una función llamada `evaluarTemperatura` que reciba como parámetro un número flotante (temperatura en grados Celsius) y retorne un entero: 0 si la temperatura es menor a 60°C (Normal), 1 si está entre 60°C y 85°C inclusive (Advertencia), y 2 si es mayor a 85°C (Peligro).
2. En la función `main`, utilice un ciclo `for` para solicitar al usuario que ingrese 5 lecturas de temperatura consecutivas.
3. Dentro del ciclo, llame a la función `evaluarTemperatura` y, usando una estructura `switch` o `if-else`, imprima por pantalla el estado del motor para cada lectura.

Ejercicio 2 – Cálculo de Resistencia Equivalente

Contexto: El cálculo de circuitos a menudo requiere iterar sobre componentes conectados en distintas configuraciones.

Instrucciones:

1. Cree una función llamada `calcularResistencia` que no reciba parámetros pero que retorne un `float`.
2. Dentro de la función, pregunte al usuario si los resistores están en Serie (ingrese 1) o en Paralelo (ingrese 2). Luego pregunte la cantidad N de resistores.
3. Utilice un ciclo `while` o `for` para solicitar el valor de cada uno de los N resistores. Emplee condicionales para aplicar la fórmula correspondiente según la configuración elegida y calcule el equivalente. Si el usuario ingresa un valor de resistencia menor o igual a cero, el programa debe mostrar un error y no contarlos en el cálculo.
4. Desde el `main`, llame a la función e imprima el resultado final.

Ejercicio 3 – Generador de Perfil de Velocidad PWM

Contexto: Para controlar la velocidad de un motor de corriente continua (DC), se utilizan señales PWM (Pulse Width Modulation). Frecuentemente se requiere hacer una 'rampa' de aceleración suave.

Instrucciones:

1. Escriba una función void `generarRampaPWM(int velocidadInicial, int velocidadFinal, int paso)`.
2. La función debe utilizar un ciclo para incrementar (o decrementar) la velocidad desde la `velocidadInicial` hasta la `velocidadFinal` sumando el valor de `paso` en cada iteración, imprimiendo el porcentaje de PWM actual.
3. Agregue una estructura condicional dentro del ciclo para asegurar que el valor de PWM impreso **nunca exceda el 100 % ni baje del 0 %**, ajustando el valor al límite correspondiente si es necesario.
4. Pruebe su función desde el `main` llamándola con valores arbitrarios, por ejemplo: inicial 20, final 120, paso 15.

Ejercicio 4 – Navegación de Robot: Gestión de Batería

Contexto: Un robot explorador debe recorrer una distancia específica, pero su avance está limitado por la capacidad de su batería. El consumo de energía varía dependiendo de la dificultad del terreno.

Instrucciones:

1. Diseñe una función int `realizarMision(int distanciaObjetivo, int bateria)` que utilice un ciclo `while`.
2. El ciclo debe ejecutarse **mientras** la distancia al objetivo sea mayor a 0 y la batería sea mayor a 0.
3. En cada iteración (que representa el avance de 1 metro), solicite al usuario que ingrese el tipo de terreno (1 para Plano, 2 para Rugoso).
4. Utilice una estructura condicional para evaluar el terreno: si es Plano, la batería disminuye en 5 unidades; si es Rugoso, disminuye en 15 unidades. Cualquier otro valor ingresado se considera un error del sensor y consume 20 unidades por penalización. Al final de la iteración, reste 1 a la distancia objetivo.
5. El ciclo termina naturalmente cuando se alcanza el objetivo o se agota la batería. La función debe retornar 1 si el robot llegó a su destino ($\text{distancia} \leq 0$) y 0 si se quedó sin energía.
6. En la función `main`, pida al usuario la distancia y la batería inicial, llame a la función y muestre si la misión fue un 'Éxito' o un 'Fracaso'.

Nota: Solución de la Clase: [Repositorio del Curso](#)